

User Manual

Motor Driver Controller MDRVCT60A002M-MCP41100-EP/ESN

REV : SEPT 2026

By Groupe Spé SI Terminale

Contents :

I – Introduction

II – Technical Specifications

- Electrical Specifications
- Schematic & Dimensions
- BOM

III – Installation and Wiring

- Power Input
- Arduino Wiring
- Driver wiring
- Status LED Reading

IV – Code example

V – File Preview

- Full Schematic
- Full PCB Layout
- Full 3D Render
- BOM

I – Introduction

The MDRVCT60A002M-MCP41100-EP/ESN is a motor driver controller that is made for [this specific 60A motor driver](#) :



It is made to replace the manual potentiometer and button so that an Arduino can pilot it. You can then integrate it in application such as RC Tank which relies on heavy motors. It works by replacing the manual potentiometer via a MCP41100, its click button and the other button via relays. Two versions exists as of this day :

- The MCP41100-EP as its PDIP interface is really easy to solder by hand.
- The MCP41100-ESN as its SOP-8 interface was easier to found online, yet is still relatively easy to solder

The PCB is publicly available but you will need to manufacture it and assemble it by yourself. The BOM will be in the corresponding GitHub release next to the PCB files.

This product is designed in a way it can pilot two of those drivers. If you only want to use one driver with it, just wire the Arduino to one MCP41100-EP and three Relays. A more detailed schematic will be available below.

[\[GitHub link to the MDRVCT60A002M-MCP41100-EP release\]](#)

[\[GitHub link to the MDRVCT60A002M-MCP41100-ESN release\]](#)

II – Technical Specification

a. Electrical Specifications

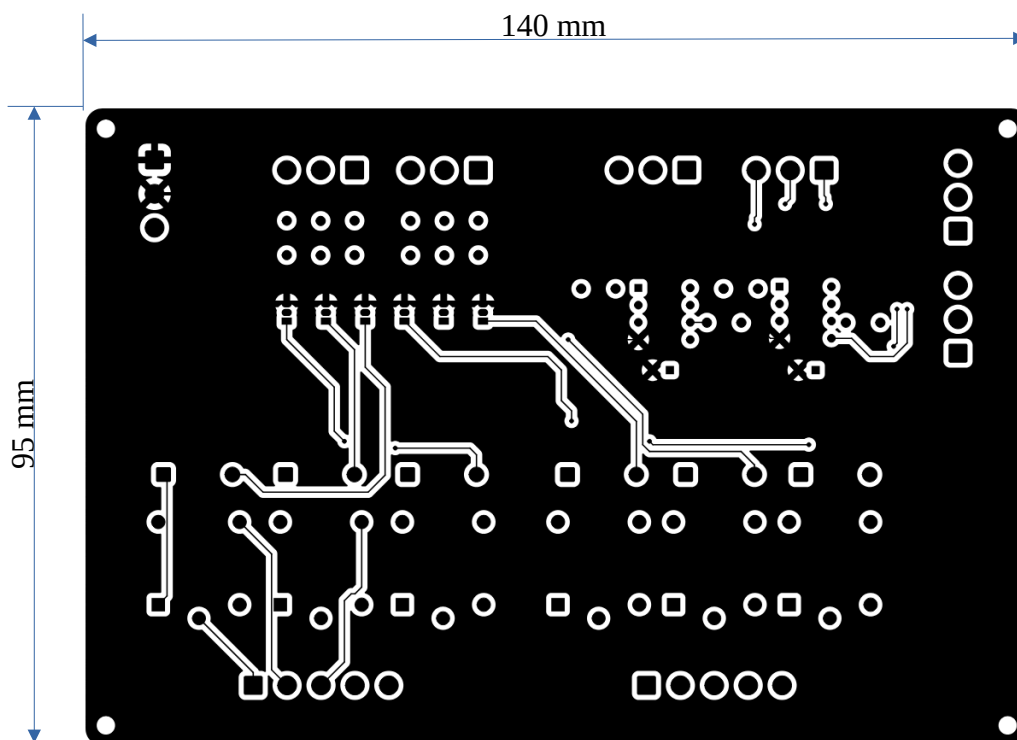
This board is made to function on 5V only, please do not input anything above or below 5V, you could fry the MCP41100-EP / ESN. If you aren't confident about your 5V input, feel free to use a LM2596 module set to 5V in order to provide a clean 5V output to the board.

The MCP41100-EP / ESN works via a SPI input interface, and its output are like a normal potentiometer (High, Low, Wiper).

When you are using it, please be sure to connect a ground that goes to your power source, and one that goes to the Arduino you are using.

b. Schematic & Dimensions

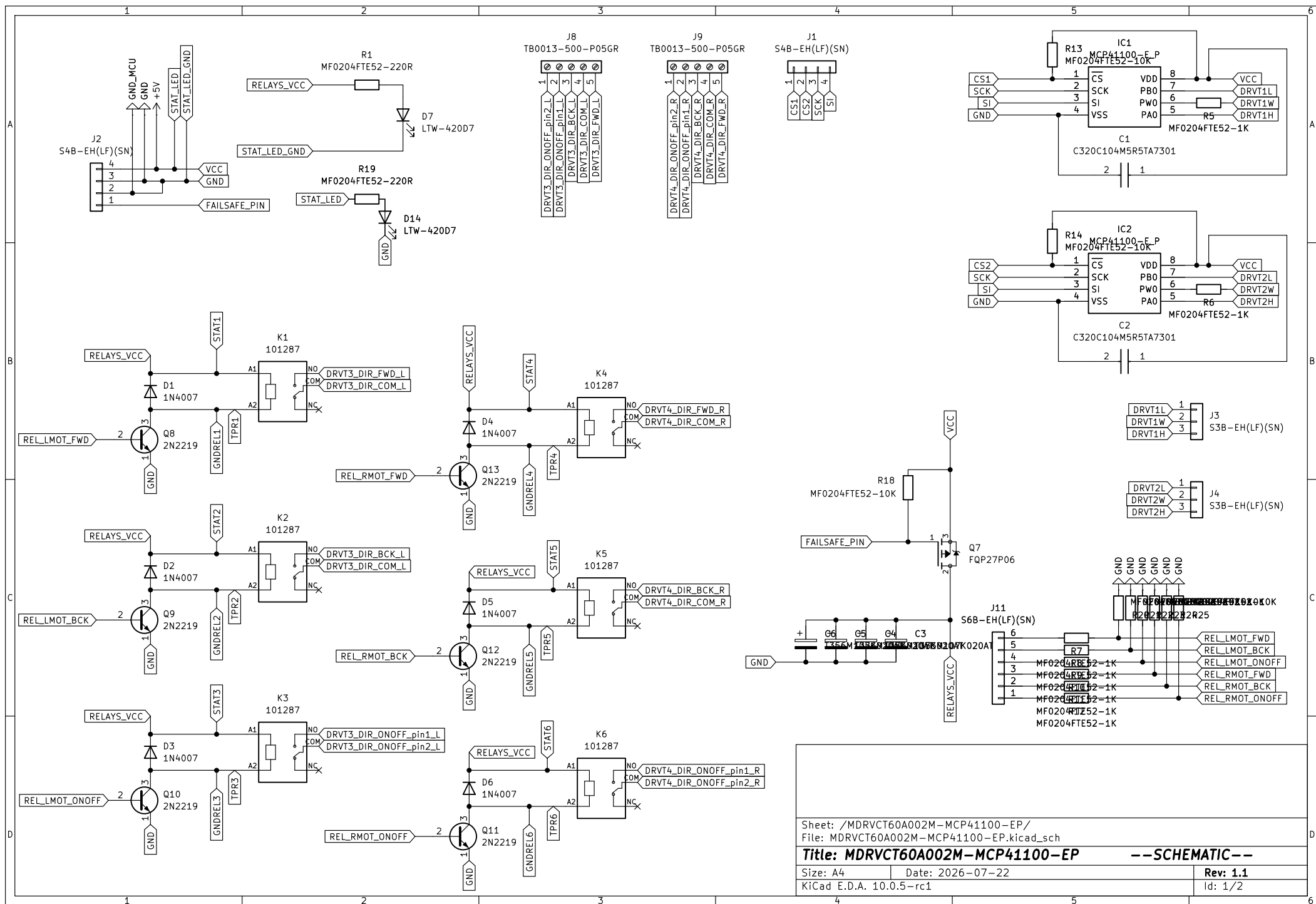
- Dimensions of the board :

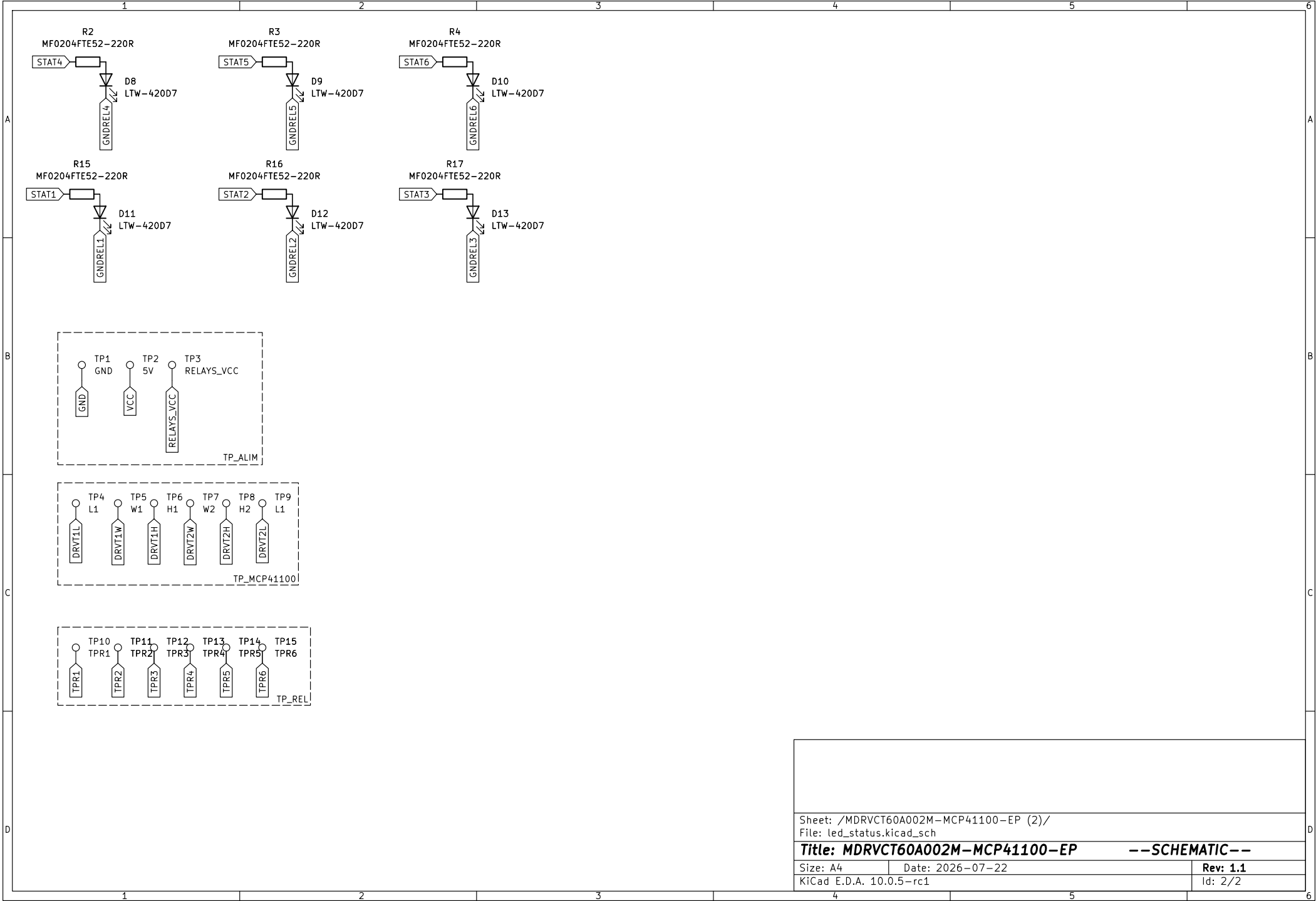


At each corner there is a mounting hole, which is an M2 mounting hole, and is placed 3.5mm away from the edges of the PCB.

Please refer to the next two pages to read the schematic

Note : There isn't any difference in the wiring between the MCP41100-EP and -ESN, the only thing that is different is the footprint used. The placement of the nearby resistors and capacitor may differ. Please choose the version of the board that suits you the best, and the component you can find online.

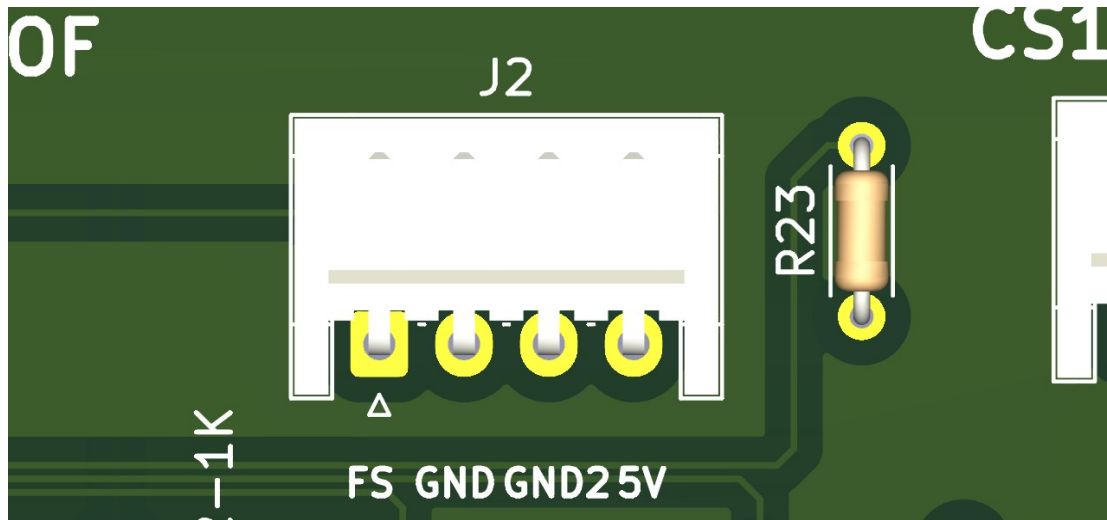




III – Installation and wiring

- Power Input

The power goes through this JST-EH connector :



From left to right pin : Failsafe (FS), GND, GND2, 5V

The 5V pin needs to be wired to a reliable power source, allowing at least 1A of current to be drawn from. Therefore the 5V Arduino PIN isn't suitable to power this board.

The GND pin is the ground that needs to return to the power source.

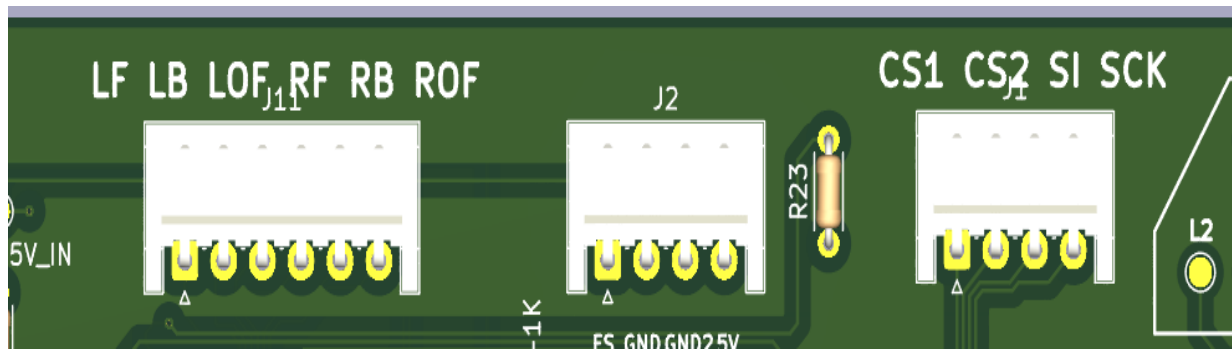
The GND2 pin needs to be wired to the micro-controller that controls the relays and the ICs.

The Failsafe pin needs to be wired to an Arduino pin which is set on LOW at all times. When the arduino loses power or disconnects, it cuts power to all relays.

- Arduino wiring

You need an Arduino, or any microcontroller that has GPIO pins and support SPI communication. Before choosing any MCU, please verify if it has enough pins to control the board (at least 11 standard I/O pins + GND).

The board is linked to the arduino via one 6-pin connector and one 4-pin connector. The first one controls all six relays while the other one controls both MCP41100s. Both connector are JST-EH type connectors. Please refer to the BOM at the end of the manual to have the full reference of the connectors. As IC1 & IC2 communicate via SPI protocol, please be sure your microcontroller has SPI pins available. For Arduino Mega, those pins are on 52 for SCK and 51 for SI (MOSI).



On the board is written what each pin controls, allowing easier testing. Both connectors are located next to each other.

Translation of each abbreviation :

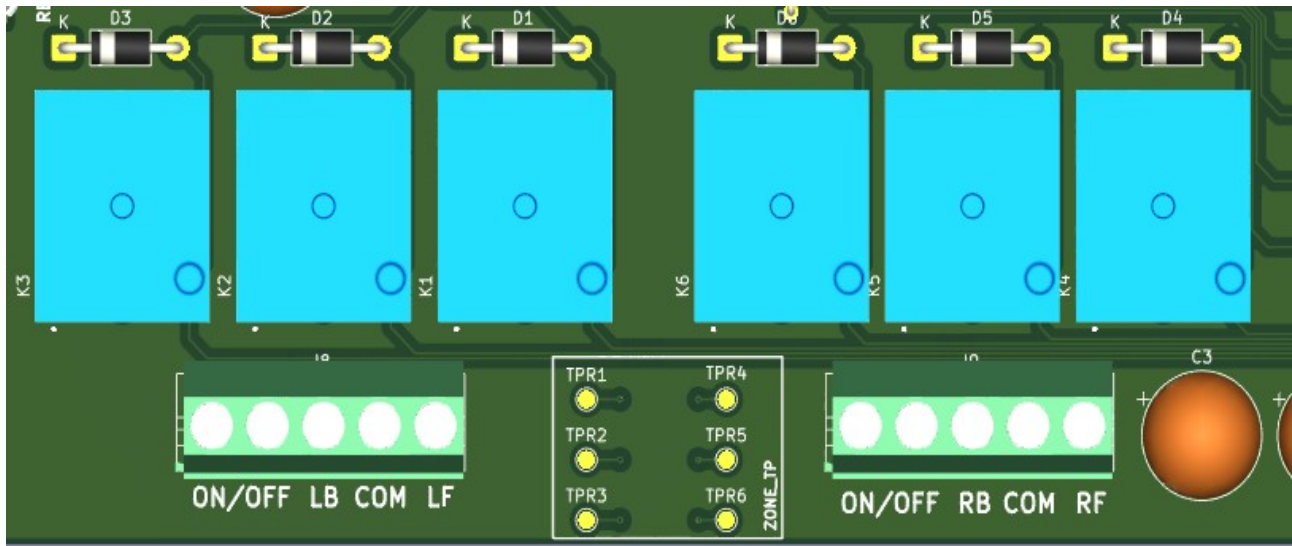
- L(R)F : Left (Right) Motor, Forward Direction. Put HIGH or LOW to enable or disable the relay.
- L(R)B : Left (Right) Motor, Backward Direction. Put HIGH or LOW to enable or disable the relay.
- L(R)OF : Left (Right) Motor, On Off Control. Put HIGH or LOW to enable or disable the relay.
- CS1 : Chip Select 1, for IC1. Put LOW to enable the chip to receive information through the SI bus.
- CS2 : Chip Select 2, for IC2. Put LOW to enable the chip to receive information through the SI bus.
- SI : Signal bus sending data to either IC1 or IC2 (MOSI). Send digital data (from 0 to 255) to control the output's resistance.
- SCK : Clock signal, operated by the microcontroller automatically.

- Driver wiring

As this board can control two drivers at the same time, there is sixteen pins on the board to send information to the drivers. But if you want to control one driver, you only need eight pins of the board.

There is two types of connector on the board that communicate to the driver :

- Screw Terminals :

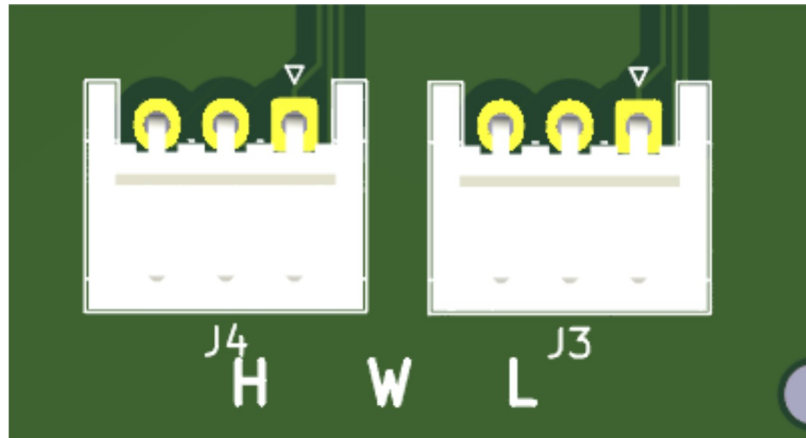


As you can see there is two 1x5 pin screw terminal. One for each driver. Explanation for the left terminal from right to left :

- 1st and 2nd pin (ON/OFF) : Left Motor On/Off control (replaces the click of the potentiometer).
- 3rd pin (LB) : Left Motor Backward Direction (replaces one of the three state button input)
- 4th pin (COM) : Left Motor three state button common pin
- 5th pin (LF) : Left Motor Forward Direction (replace ones of the three state button input)

The same logic applies to the right terminal, but for the right motor.

- JST-EH Type Connectors :



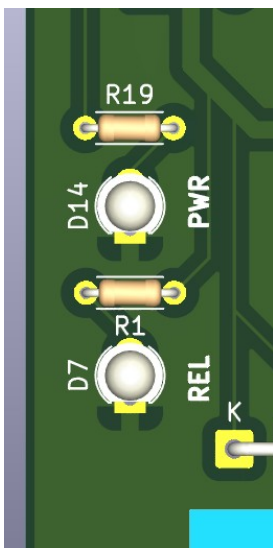
Then there is two 3-pins EH-Type connectors. They are used as a replacement to the manual potentiometer. There is one connector for each driver. And both work the same (from left to right) :

- L : Low Terminal
- W : Wiper Pin
- H : High Terminal

- Status LEDs Reading

There are 8 LEDs that act as status indication of the board.

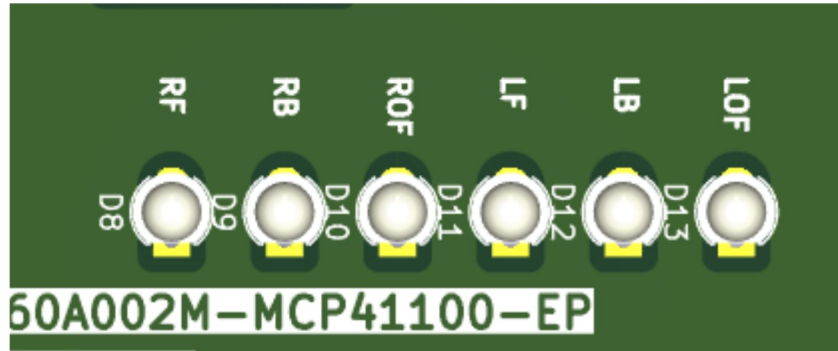
The first set of LEDs is located on the left of the board and the LEDs are labeled « PWR » and « REL » :



The « PWR » LED indicates whether the board is electrically powered or not, even if the Microcontroller is unresponsive or don't send any information through the communication ports, it will be on as long as power is connected to the board.

The « REL » LED indicates whether all 6 relays are connected to power or not. It is connected to the failsafe MOSFET and will be on if power is allowed to go through the relays, meaning that the FS pin is set LOW by the microcontroller.

The second set of LEDs is located on the down-right of the board, and the LEDs are labeled « LOF », « LB », « LF », « ROF », « RB » and « RF ». Each of these is connected to one relay and is on if the relay is set open, if any of them is off, it means that no current is applied to the relay, therefore the relay is closed.



IV – Code example

This code applies to an RC Tank, and is therefore controlling two drivers via the MDRVCT60A002M-MCP41100-EP/ESN, and via an RC controller. However it isn't optimised, it is just an example about the board works.

```
#include <Arduino.h>
#include <SPI.h>

// =====
// 1. PIN DEFINITIONS
// =====

// RC Controller Input Pins
#define speedPin A9
#define steeringPin A8

// Left Motor Relays
#define leftMotorFwd 22
#define leftMotorBck 23
#define leftMotorPower 26

// Right Motor Relays
#define rightMotorFwd 24
#define rightMotorBck 25
#define rightMotorPower 27

// Digital Potentiometer SPI Pins (Chip Select)
#define leftPotCS 42
#define rightPotCS 43

// Relay logic (Low-level trigger)
#define RELAY_ON LOW
#define RELAY_OFF HIGH

// =====
// 2. SET MOTOR POWER
// =====
// This function sends a raw value (0 to 255) to the digital
// potentiometer via SPI to control the motor controller's throttle.

void setMotorSpeed(int csPin, int speedValue) {
    digitalWrite(csPin, LOW); // Select the specific potentiometer
    SPI.transfer(0x11);       // Send the "Write Data" command
    SPI.transfer(speedValue); // Send the speed level (0 to 255)
    digitalWrite(csPin, HIGH); // Deselect the potentiometer
}
```

```

// =====
// 3. SETUP: INITIALIZE PINS & SPI
// =====
void setup() {
    pinMode(speedPin, INPUT);
    pinMode(steeringPin, INPUT);

    pinMode(leftMotorFwd, OUTPUT);
    pinMode(leftMotorBck, OUTPUT);
    pinMode(leftMotorPower, OUTPUT);
    pinMode(rightMotorFwd, OUTPUT);
    pinMode(rightMotorBck, OUTPUT);
    pinMode(rightMotorPower, OUTPUT);

    digitalWrite(leftMotorFwd, RELAY_OFF);
    digitalWrite(leftMotorBck, RELAY_OFF);
    digitalWrite(rightMotorFwd, RELAY_OFF);
    digitalWrite(rightMotorBck, RELAY_OFF);

    digitalWrite(leftMotorPower, RELAY_ON);
    digitalWrite(rightMotorPower, RELAY_ON);

    pinMode(leftPotCS, OUTPUT);
    pinMode(rightPotCS, OUTPUT);
    digitalWrite(leftPotCS, HIGH);
    digitalWrite(rightPotCS, HIGH);
    SPI.begin();
}

// =====
// 4. MAIN LOOP: READ & DRIVE
// =====
void loop() {
    // A standard RC signal is between 980 (min) and 2000 (max), with ~1500
    // being the center.
    int rawSpeed = pulseIn(speedPin, HIGH);
    int rawSteering = pulseIn(steeringPin, HIGH);

    int speedPercentage = 0;
    int steeringPercentage = 0;

    // Deadzone
    if (abs(rawSpeed - 1500) > 50) {

```

```

    speedPercentage = map(rawSpeed, 980, 2000, -100, 100);
}
if (abs(rawSteering - 1500) > 50) {
    steeringPercentage = map(rawSteering, 980, 2000, -100, 100);
}

// Differential Steering
int leftSpeed = speedPercentage;
int rightSpeed = speedPercentage;

if (steeringPercentage > 0) { // Turning Right
    leftSpeed = speedPercentage + steeringPercentage;
    rightSpeed = speedPercentage - steeringPercentage;
} else if (steeringPercentage < 0) { // Turning Left
    leftSpeed = speedPercentage - abs(steeringPercentage);
    rightSpeed = speedPercentage + abs(steeringPercentage);
}

leftSpeed = constrain(leftSpeed, -100, 100);
rightSpeed = constrain(rightSpeed, -100, 100);

if (leftSpeed > 0) {
    digitalWrite(leftMotorFwd, RELAY_ON);
    digitalWrite(leftMotorBck, RELAY_OFF);
} else if (leftSpeed < 0) {
    digitalWrite(leftMotorFwd, RELAY_OFF);
    digitalWrite(leftMotorBck, RELAY_ON);
} else {
    digitalWrite(leftMotorFwd, RELAY_OFF);
    digitalWrite(leftMotorBck, RELAY_OFF);
}
// Convert 0-100% to 0-255 for the SPI chip
setMotorSpeed(leftPotCS, map(abs(leftSpeed), 0, 100, 0, 255));

if (rightSpeed > 0) {
    digitalWrite(rightMotorFwd, RELAY_ON);
    digitalWrite(rightMotorBck, RELAY_OFF);
} else if (rightSpeed < 0) {
    digitalWrite(rightMotorFwd, RELAY_OFF);
    digitalWrite(rightMotorBck, RELAY_ON);
} else {
    digitalWrite(rightMotorFwd, RELAY_OFF);
    digitalWrite(rightMotorBck, RELAY_OFF);
}
setMotorSpeed(rightPotCS, map(abs(rightSpeed), 0, 100, 0, 255));

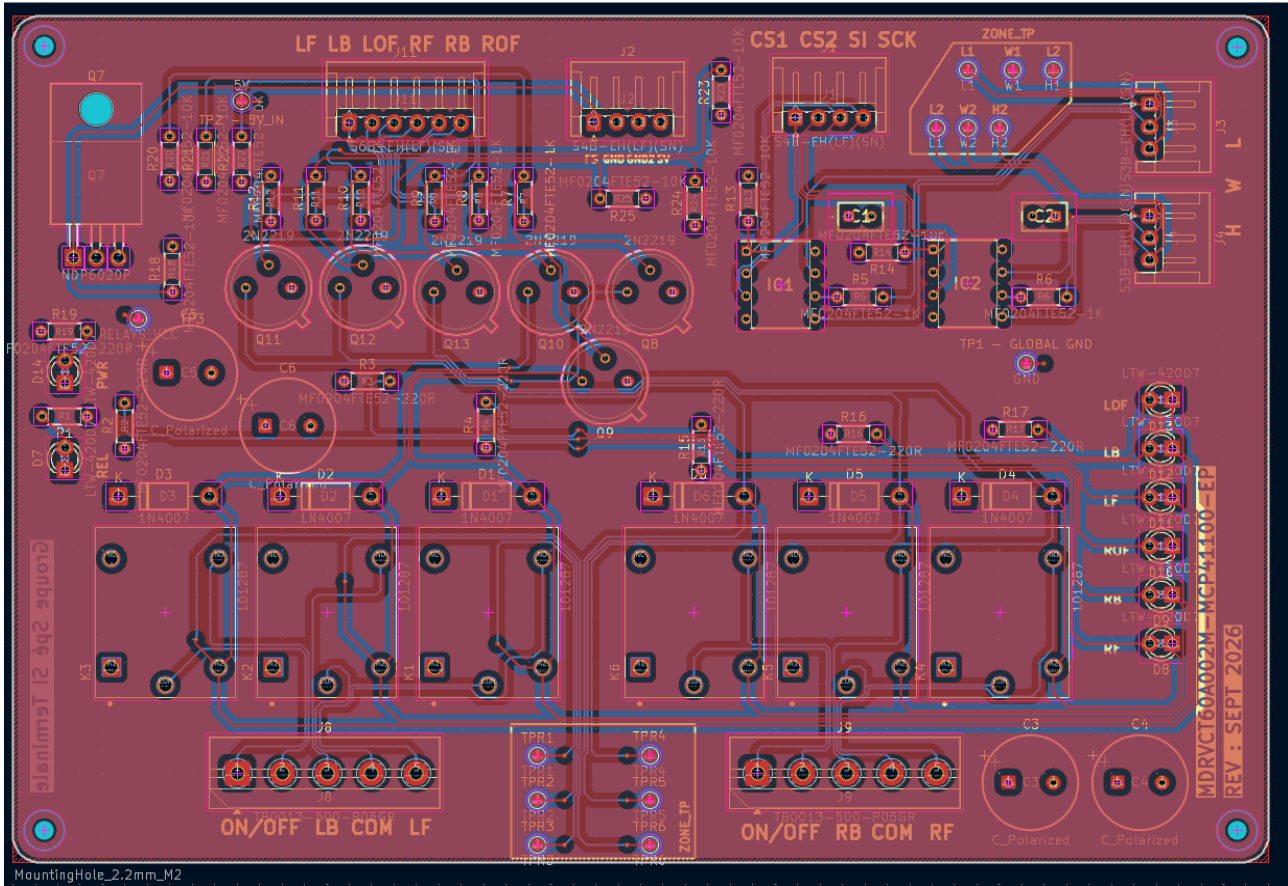
// Delay to prevent destructive relay switching
delay(200);
}

```

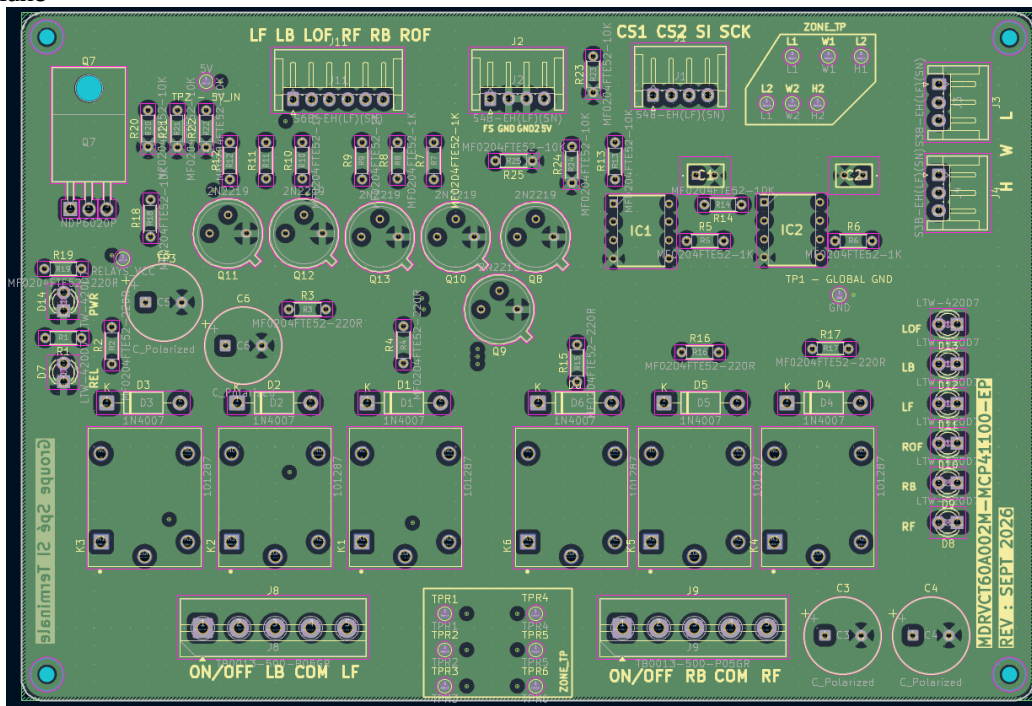
V – File Preview

- PCB Layout (MCP41100-EP)

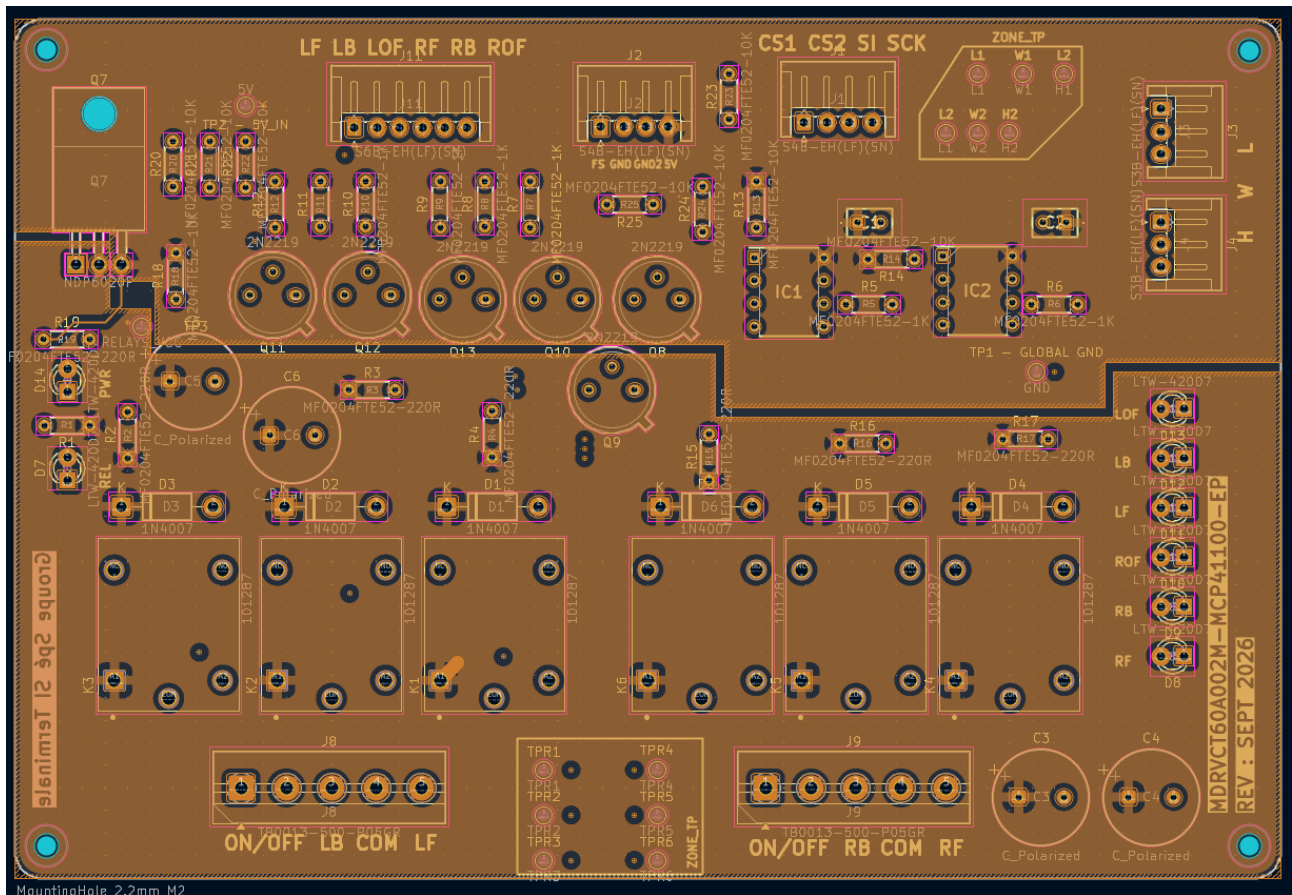
→ Front and Back layer



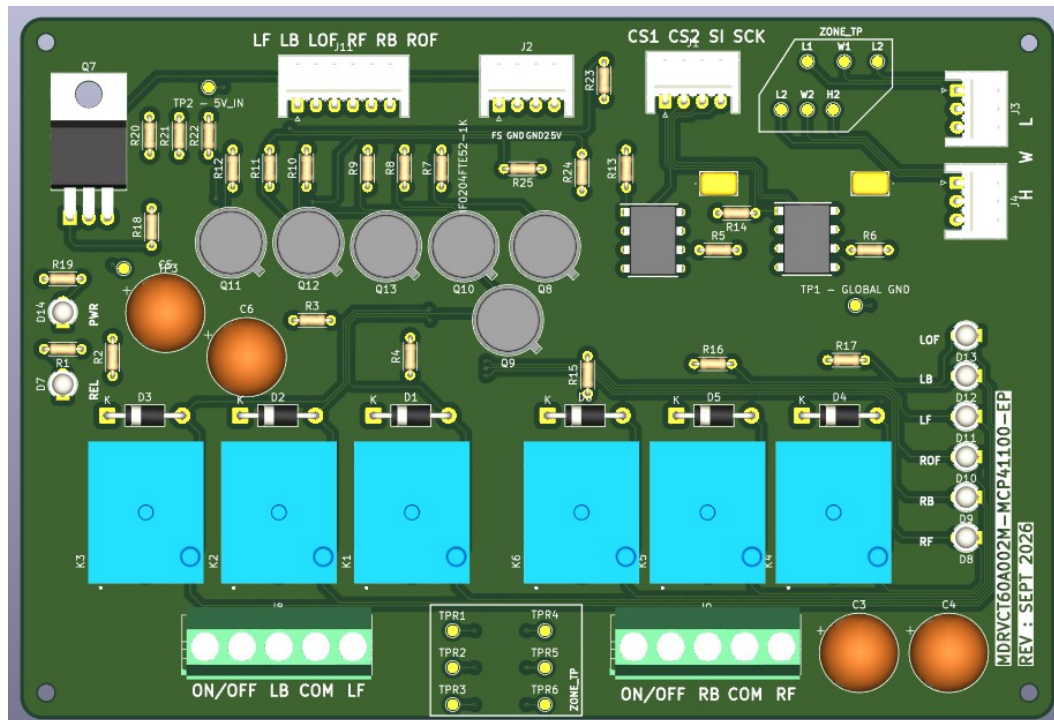
→ GND Plane



→ 5V Power Plane (Board and Relay Power)

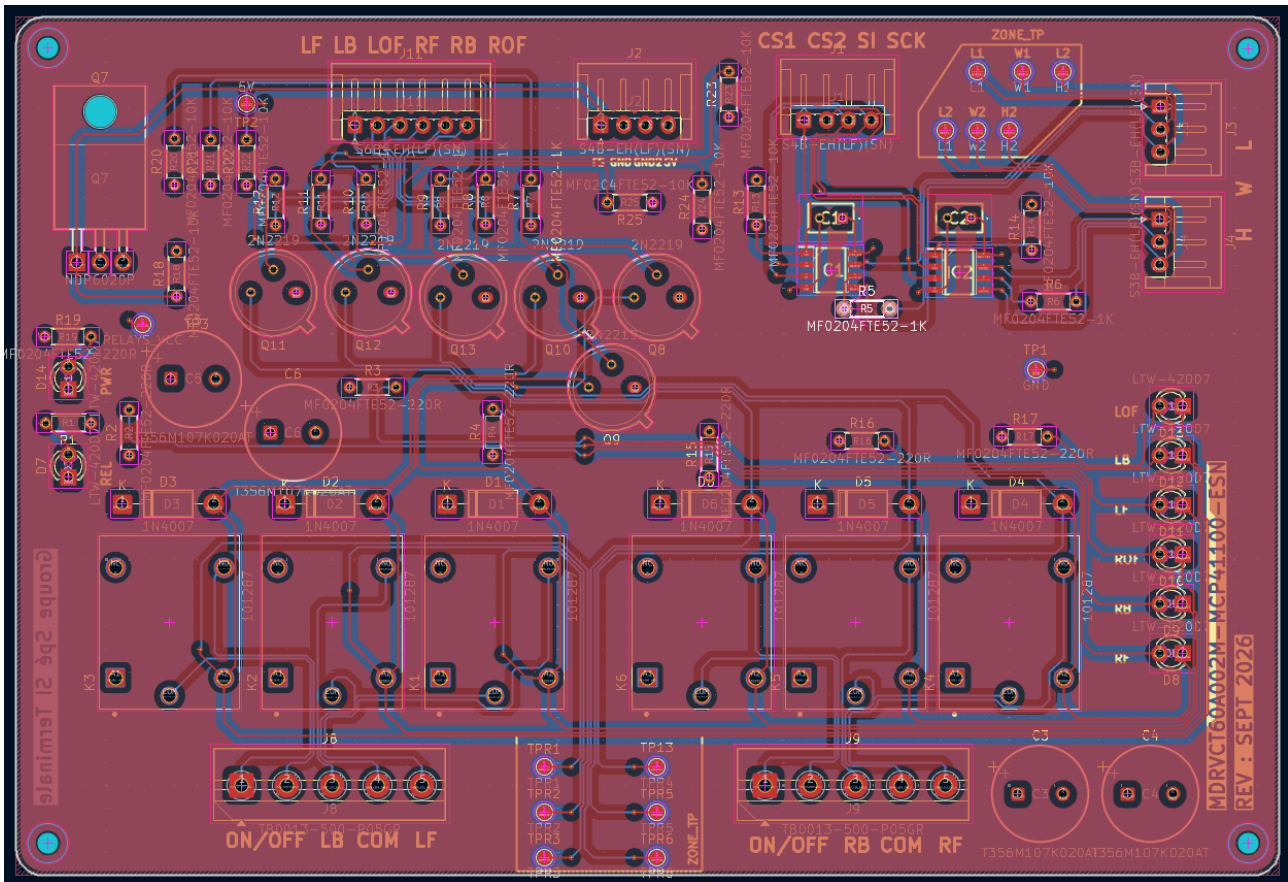


- 3D Render

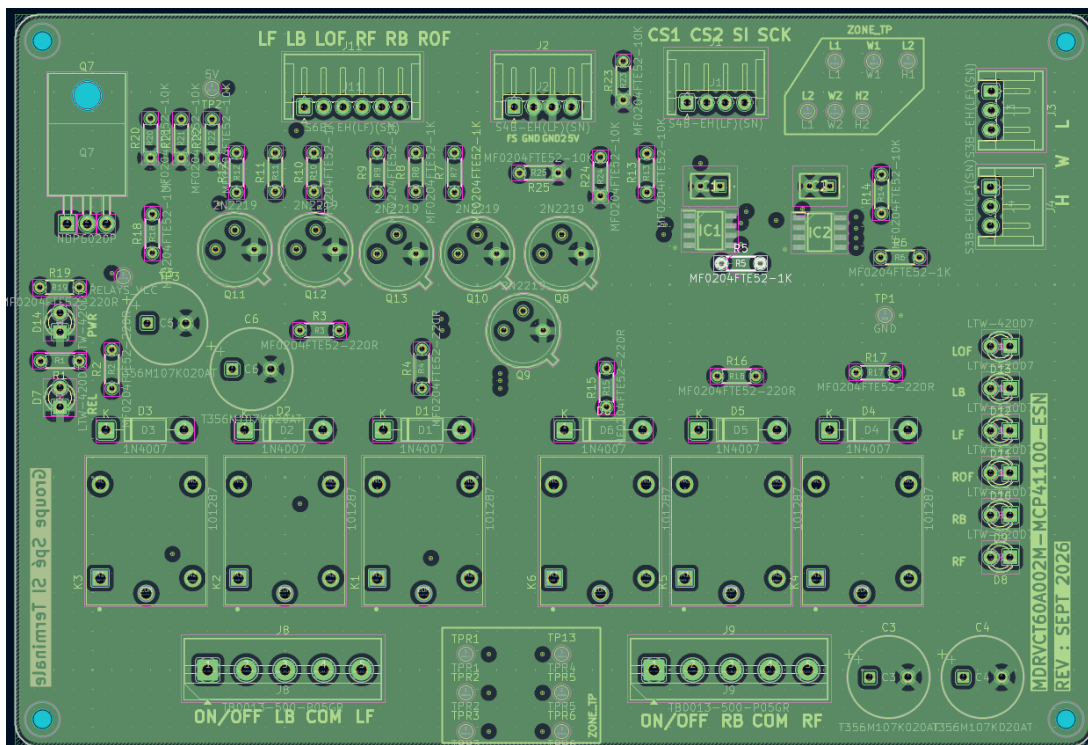


- PCB Layout (MCP41100-ESN)

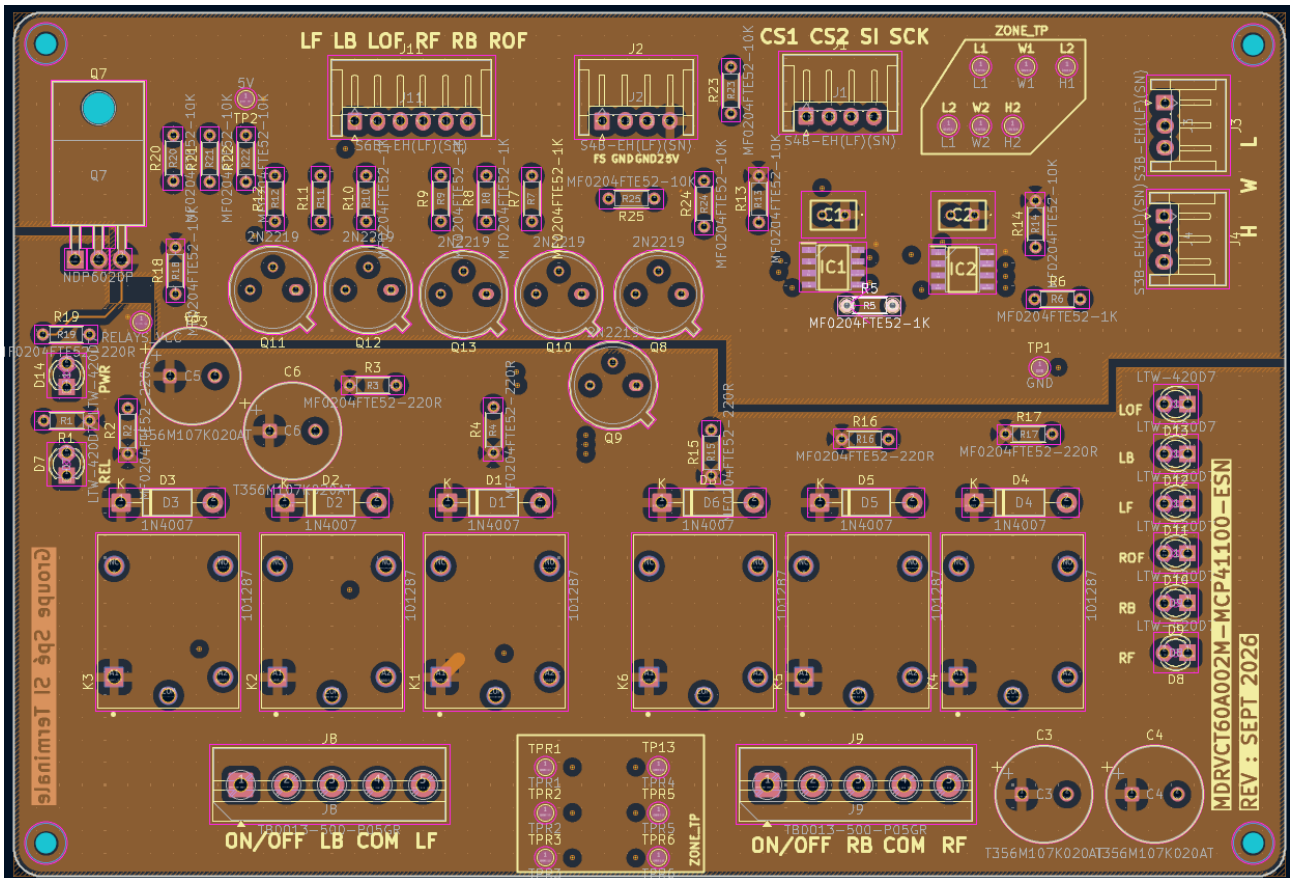
→ Front and back layer



→ GND Plane



→ 5V Power Plane (Board and Relay Power)



→ 3D Render

